

AI BlogNest: AI-Powered Blog Content Management System

Final Project Documentation

Project Name	AI-Augmented Backend Development: AI BlogNest API
Team ID	SWTID-2026-3024
Project Version	1.0.0
Documentation Date	30 September 2026
Submitted By	[Student Name / Team Lead]
Register No.	[Register Number]
Program / Semester	[Program, Semester]
Project Guide	[Guide Name], Assistant Professor
Department	Department of Computer Applications

1. Abstract

Modern content platforms need a backend that is secure, organised and able to serve different kinds of clients. AI BlogNest is a RESTful backend built with Node.js, Express.js and MongoDB. It lets users register, log in, write and manage blog posts, and comment on them. It also connects to Google Gemini AI so that blog drafts can be generated and long articles can be summarised automatically. Access is controlled with JWT authentication and four user roles: Admin, Editor, Author and Reader.

This document combines every phase of the project: problem definition, ideation, requirements, architecture, data flow, user stories, database design, security, implementation, AI integration and testing.

2. Introduction and Problem Statement

2.1 Background

Creating, organising, updating and summarising blog content by hand takes a lot of time. Content creators also need secure accounts and proper control over who can do what. Most traditional blog systems depend on manual writing and have no built-in AI help.

2.2 Problem Statements

ID	I am	I am trying to	But	Because	Which makes me feel
PS-1	A content creator or blogger	Create and manage high-quality blog content efficiently	Writing and managing content manually needs a lot of time and effort, especially for many articles and summaries	Traditional blog systems depend on manual creation and organisation	Time-consuming and less efficient
PS-2	A blog platform user or content manager	Manage content activities in one place	Handling different activities separately makes the process difficult and complicated	There is a need for centralised management, secure authentication, role-based access and AI-assisted processing	Less efficient while managing regular content work

2.3 Case Study

Priya is a digital content creator who runs several niche blogs. As her work grew, she faced these problems:

Challenge	Impact
Draft organisation	Duplicate content and missing media paths across revisions
Access control	No separation of privileges, so authors could change global settings
Comment moderation	Spam comments piled up because there was no central filter

Challenge	Impact
Content bottlenecks	Summarising long articles for social media and meta tags took too long

2.4 Proposed Solution

AI BlogNest provides one central REST API that handles authentication, role-based permissions, blog management, comment management and AI-based content generation and summarisation. Because it is a backend API, any frontend or tool such as Postman can use it.

2.5 Objectives

- Provide secure registration and login using JWT and password hashing.
- Give each role only the permissions it needs.
- Support full create, read, update and delete operations for blogs.
- Generate blog drafts and short summaries using Google Gemini AI.
- Store all data safely in MongoDB.
- Return clear responses for both valid and invalid requests.

2.6 User Personas

Persona	Usage
Author	Creates drafts, adds tags and media links, checks view counts and feedback
Editor / Admin	Reviews drafts, publishes posts, moderates comments, upgrades user roles
Reader	Reads published posts, searches tags, comments after logging in
AI Automation	Triggers Gemini to write drafts or shorten long text into summaries

3. Ideation Phase

3.1 Team Discussion

The team met to discuss problems in digital content creation and blog management. The selected problem was: *to develop an AI-powered blog content management system that helps users create, manage, organise and summarise blog content with secure authentication and role-based access.*

3.2 Brainstormed Ideas (Grouped)

Group	Ideas
AI Features	Blog content generation, content summarisation, content assistance
Blog Management	Create, view, update, delete posts; manage comments and categories
User Management	Registration, login, authentication, role-based access
Database	Store users, blogs and comments in MongoDB
Security	JWT, bcrypt.js hashing, input validation, sanitisation, error handling
Testing	REST API testing, Postman testing, response verification

3.3 Idea Prioritisation

Priority	Idea	Reason
1	AI Blog Generation	Helps users produce content efficiently
2	User Authentication	Gives secure access
3	Blog Management	Lets users create and manage posts
4	Role-Based Access	Different permissions for different users
5	MongoDB Database	Stores and manages application data

Priority	Idea	Reason
6	AI Summarisation	Creates short summaries of long posts
7	Comment Management	Manages comments on blogs
8	Security and Validation	Protects the app and checks user input
9	API Testing	Verifies that backend APIs work

Final selected idea: AI BlogNest, an AI-Powered Blog Content Management System.

4. System Requirements

4.1 Software Requirements

Component	Specification	Role
Operating System	Ubuntu 22.04 LTS / Windows 11 / macOS	Host system
Runtime	Node.js v18.x or v20.x LTS	JavaScript runtime
Web Framework	Express.js v4.18+	Routing and middleware
Database	MongoDB Atlas / MongoDB v6.0+	NoSQL document store
ODM	Mongoose v7.x+	Schemas and models
Testing Tools	Postman / Thunder Client	API testing
Editor	Visual Studio Code	Development

4.2 Hardware Requirements

Unit	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 3.0 GHz or higher
RAM	4 GB	16 GB
Disk	1 GB free	20 GB SSD
Network	100 Mbps	1 Gbps

4.3 Non-Functional Requirements

Attribute	Requirement	How it is met
Availability	99.9% uptime goal	MongoDB Atlas failover and health endpoints
Scalability	Horizontal scaling	Stateless JWT works with load balancing
Maintainability	Modular MVC code	Separate controllers, services and routes
Performance	Average API latency under 100 ms	Indexed queries and lean responses
Security	OWASP Top 10 awareness	Hashing, token expiry, sanitisation

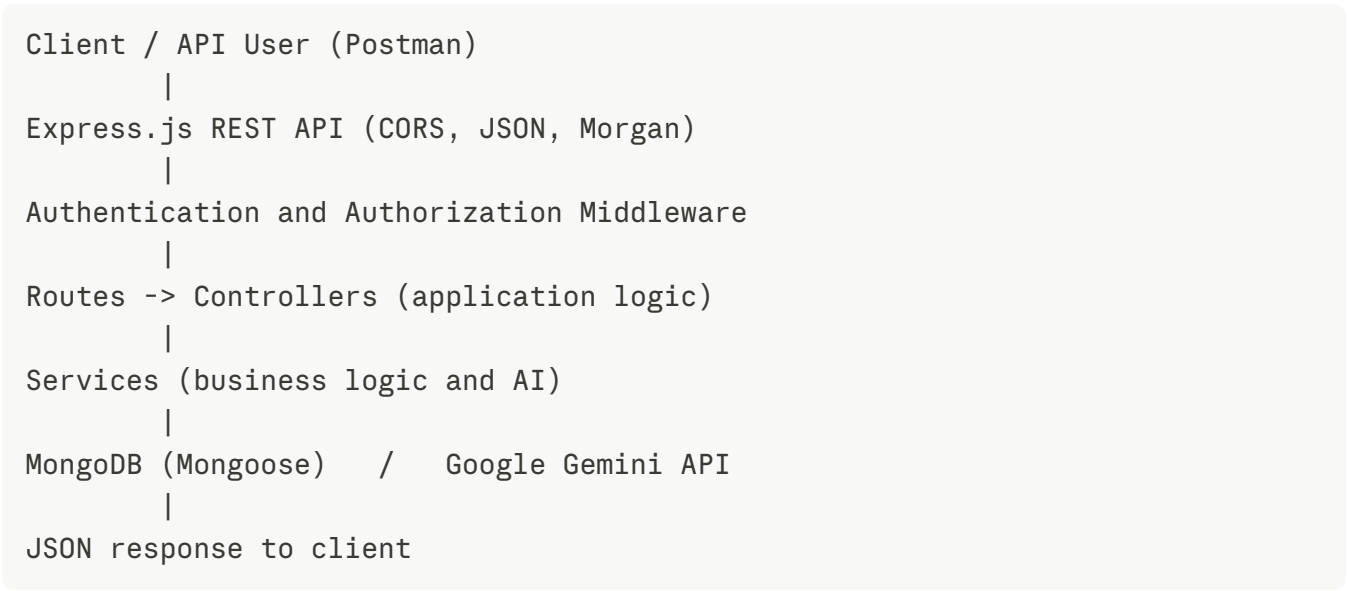
4.4 Main Dependencies

Package	Version	Purpose
jsonwebtoken	9.0.2	JWT creation and verification
bcryptjs	2.4.3	Password hashing
@google/generative-ai	0.1.1	Google Gemini SDK
dotenv	16.3.1	Environment variables
morgan	1.10.0	HTTP request logging
cors	2.8.5	Cross-origin requests

5. Technology Stack and Architecture

5.1 Layered Architecture

The backend follows a layered design. Every request passes through the same steps in order:



5.2 Components and Technologies

No.	Component	Description	Technology
1	User Interface	Users interact through REST requests and testing tools	REST API, Postman, JSON
2	Application Logic 1	Routing, request processing, server logic	Node.js, Express.js
3	Application Logic 2	Authentication, password protection, role control	JWT, bcrypt.js, Express middleware
4	Application Logic 3	AI blog generation and assistance	Google Gemini API
5	Database	Users, blogs, comments, categories, tags	MongoDB, Mongoose
6	Cloud Database	Cloud hosting and remote access	MongoDB Atlas
7	File Storage	Project configuration and application files	Local file system
8	External API	AI content generation	Google Gemini API
9	AI Model	AI-assisted content and summaries	Google Gemini generative model

No.	Component	Description	Technology
10	Infrastructure	Runs the backend and database	Node.js local server, MongoDB Atlas

5.3 Application Characteristics

Characteristic	Description
Open-source frameworks	Node.js, Express.js, Mongoose and MongoDB are used
Security	JWT, hashing, role-based access, validation, sanitisation, rate limiting
Scalable design	API routing, auth, controllers, business logic, database and AI are separate modules
Availability	Can run on a continuous server, with Atlas giving cloud database availability
Performance	Light JSON communication, indexed and efficient queries, validation

5.4 Local and Cloud Split

Local	Cloud
Node.js runtime, Express server, VS Code, Postman, project files (port 5000)	MongoDB Atlas (database), Google Gemini API (AI)

5.5 Architecture Styles Used

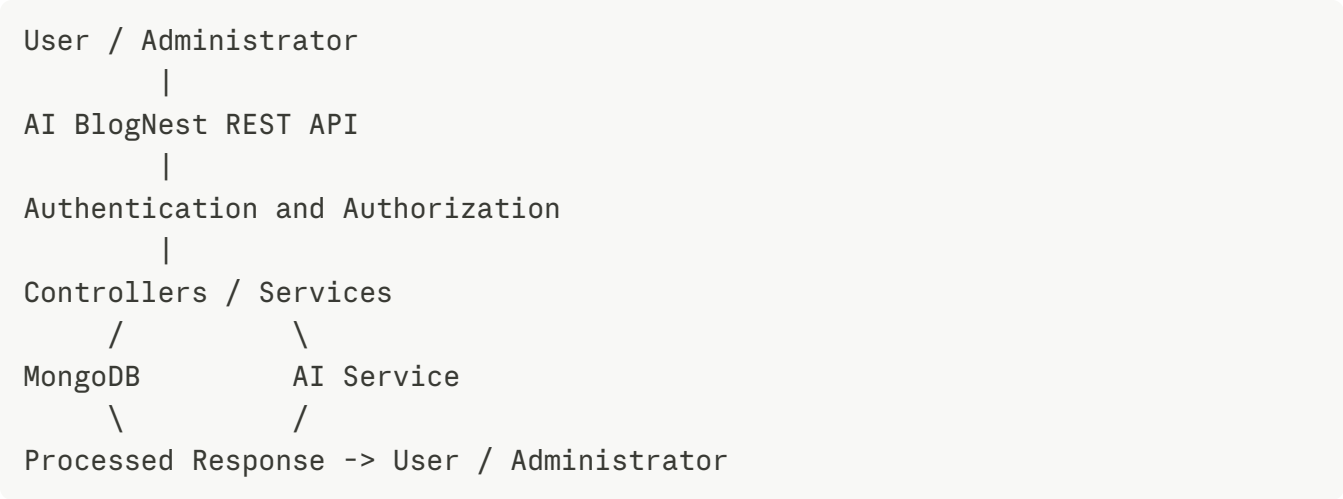
- **Modular:** authentication, blogs, AI and database code are in separate modules.
- **RESTful:** GET, POST, PUT/PATCH and DELETE are used for matching operations.
- **Database-driven:** MongoDB stores data and Mongoose defines schemas.
- **AI-integrated:** the backend sends prompts to Gemini and returns the result.
- **Secure:** bcrypt.js hashes passwords and JWT protects endpoints.
- **Cloud-ready:** Atlas makes future deployment and scaling easier.

6. Data Flow

6.1 External Entities

Entity	Interaction
User / Content Creator	Sends registration, login, blog, comment, AI prompt and summary requests; receives responses
Administrator	Manages users, roles and content activities
AI Service (Gemini)	Receives prompts from the backend and returns generated content or summaries
MongoDB Database	Stores users, blogs, comments, categories and tags

6.2 Main Flow



6.3 Processes

No.	Process	What it does
1	User Authentication	Receives and validates details, hashes passwords, issues JWT, stores users
2	User Authorization	Verifies the token, finds the role, allows or denies access

No.	Process	What it does
3	Blog Management	Creates, reads, updates and deletes posts; manages categories and tags
4	Comment Management	Receives comments and links them to blog posts
5	AI Content Generation	Takes a topic, sends it to Gemini, returns the generated draft
6	AI Summarisation	Sends existing content to Gemini and returns a short summary
7	Database Management	Stores and retrieves all records using Mongoose

7. User Stories

Story	User Type	Epic	User Story	Acceptance Criteria	Priority	Release
USN-1	Content Creator	Registration and Authentication	I can register by giving the required details	A valid submission creates a new account	High	Sprint 1
USN-2	Content Creator	Registration and Authentication	I can log in using my email and password	Valid credentials give a successful login and token	High	Sprint 1
USN-3	Content Creator	Registration and Authentication	My password is stored using hashing	Password is never stored as plain text	High	Sprint 1
USN-4	Content Creator	Authentication	I receive a JWT token to access protected APIs	A valid token is generated after login	High	Sprint 1
USN-5	Administrator	Role Management	I can manage user roles and permissions	Users get proper roles and cannot do unauthorised actions	High	Sprint 1
USN-6	User	Security and Validation	I get a clear response when invalid data is sent	Invalid or incomplete data is rejected with a message	High	Sprint 1

Story	User Type	Epic	User Story	Acceptance Criteria	Priority	Release
USN-7	User	Security and Validation	I cannot reach protected APIs without authentication	Unauthorised requests are rejected	High	Sprint 1

Other requirements verified during testing:

- Authorised users can create and manage blog posts and retrieve blog content.
- Users can only perform operations allowed for their role.
- Content creators can generate blog content using AI.
- Users can get a short summary of existing content.
- User and blog data is stored securely in MongoDB.
- The system returns proper responses for valid and invalid requests.

8. Database Design

MongoDB stores the data and Mongoose defines the schemas. Collections are linked using `ObjectId` references.

8.1 Entities

Entity	Main Fields
User	<code>_id</code> , name, email (unique), password (hashed), avatar, role (Admin / Editor / Author / Reader), createdAt, updatedAt
Blog Post	<code>_id</code> , title, slug (unique), content, category, author (ref to User), authorName, status (Draft / Published), createdAt, updatedAt
Comment	<code>_id</code> , content, status (Pending / Approved / Spam), author (ref to User), blog (ref to Blog), createdAt

Relationships: one User has many Blog Posts, and one Blog Post has many Comments.

8.2 Indexes

Indexes avoid scanning the whole collection on every query:

- `UserSchema.index({ email: 1 }, { unique: true })` for fast login lookup.
- `BlogSchema.index({ slug: 1 }, { unique: true })` for fetching posts by URL slug.
- `BlogSchema.index({ category: 1, createdAt: -1 })` for category lists, newest first.

9. Authentication and Security

The system uses stateless JSON Web Tokens. After login, the server signs a token, and the client sends it with every protected request.

9.1 Request Verification Steps

1. The client sends `Authorization: Bearer <token>`.
2. `authMiddleware` reads the header.
3. `jwt.verify()` checks the token using `JWT_SECRET`.
4. The user is loaded from MongoDB and attached to `req.user`.
5. A role check such as `checkRole(['admin', 'editor'])` runs for higher-level routes.
6. If the token is missing or expired, the server returns **401 Unauthorized**.

9.2 Security Controls

Area	Threat	Protection
Password storage	Rainbow table attacks	bcrypt salted hashing with 10 salt rounds
Token misuse	Replay attacks	JWT expiry of 24 hours and a secret key in the environment file
NoSQL injection	Data leakage	Mongoose type enforcement and query escaping
Bad input	XSS and malformed data	Validation and sanitisation (express-validator)
Abuse	Too many requests	Rate limiting

Area	Threat	Protection
Errors	Server crashes	Central error-handling middleware

10. Project Structure and Implementation

10.1 Directory Structure

```
AI-BLOGNEST/  
|-- node_modules/  
|-- src/  
|   |-- config/  
|   |   |-- db.js           MongoDB connection  
|   |-- controllers/  
|   |   |-- authController.js  registration and JWT  
|   |   |-- blogController.js  blog CRUD and filters  
|   |   |-- aiController.js    Gemini bridge  
|   |-- middleware/  
|   |   |-- authMiddleware.js  token check and role guard  
|   |   |-- errorMiddleware.js global error handler  
|   |-- models/  
|   |   |-- User.js  
|   |   |-- Blog.js  
|   |-- routes/  
|   |   |-- authRoutes.js      /api/auth  
|   |   |-- blogRoutes.js      /api/blogs  
|   |   |-- aiRoutes.js        /api/ai  
|   |-- services/  
|   |   |-- geminiService.js   Gemini SDK interface  
|   |-- app.js                 Express setup  
|   |-- server.js              entry point  
|-- .env                       environment variables  
|-- package.json
```

10.2 User Model (models/User.js)

```
const mongoose = require('mongoose');  
const bcrypt = require('bcryptjs');  
  
const UserSchema = new mongoose.Schema({
```

```

    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    password: { type: String, required: true, minlength: 6 },
    role: {
      type: String,
      enum: ['Admin', 'Editor', 'Author', 'Reader'],
      default: 'Author'
    }
  }, { timestamps: true });

// hash the password before saving, only if it was changed
UserSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

module.exports = mongoose.model('User', UserSchema);

```

10.3 Server Entry Point (`server.js`)

```

const dotenv = require('dotenv');
dotenv.config();

const app = require('./app');
const connectDB = require('./config/db');

connectDB(); // connect to MongoDB Atlas

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`[BlogNest] Server running on port: ${PORT}`);
});

```

10.4 Request Lifecycle

1. **Ingress:** Express applies CORS and parses the JSON body into `req.body`.
2. **Authentication:** `authMiddleware` checks the JWT.
3. **Processing:** the controller runs the logic, using Mongoose models or the Gemini service.

4. **Response:** a structured JSON reply is returned with a status code (200, 201, 400, 401, 404, 500).

11. Generative AI Integration (Google Gemini)

The AI features live in `services/geminiService.js`. The controller first validates the input, then calls the service, which sends a prompt to Gemini and returns the text.

```

Client prompt (topic and category)
    |
/api/ai/generate-blog
    |
Controller validation
    |
geminiService.js
    |
Google Gemini model
    |
Generated content -> JSON response

```

11.1 AI Service Code (`services/geminiService.js`)

```

const { GoogleGenerativeAI } = require('@google/generative-ai');

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

// write a full blog post for a topic and category
exports.generateBlogContent = async (topic, category) => {
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const prompt = `Write a comprehensive technical blog post about "${topic}"
in the "${category}" category. Include introduction, key concepts, and
conclusion.`;
  const result = await model.generateContent(prompt);
  const response = await result.response;
  return response.text();
};

// shorten an article into three sentences
exports.summarizeContent = async (text) => {
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const prompt = `Provide a concise 3-sentence summary for the following

```

```
    article content: `${text}`;  
    const result = await model.generateContent(prompt);  
    const response = await result.response;  
    return response.text();  
};
```

The model name is set in one place, so it can be changed easily if Google updates its available models.

11.2 Error Handling for AI Calls

Scenario	Failure	Handling
API quota exceeded	HTTP 429	Fall back to a standard cached summary response
Invalid input	HTTP 400	Validation middleware blocks the call before the SDK runs
Network problem	Timeout	Retry with backoff, up to 3 attempts

12. Testing and User Acceptance Testing (UAT)

12.1 Scope

Registration, login, JWT authentication, role-based access, blog create / read / update / delete, comments, AI generation, AI summarisation, MongoDB operations, validation, error handling and API response checks.

12.2 Test Environment

Item	Detail
URL	http://localhost:5000 (local development server)
API Tools	Postman and Thunder Client
Backend	Node.js with Express.js
Database	MongoDB with Mongoose
AI Service	Google Gemini AI

Item	Detail
Credentials	Provided through the API during testing; passwords are never stored in plain text

12.3 Test Cases

ID	Scenario	Expected Result	Actual Result	Status
TC001	Registration with valid details	Account created and stored in MongoDB	Account created, success response received	Pass
TC002	Registration with missing fields	Request rejected with validation error	Validation message returned	Pass
TC003	Registration with duplicate email	Duplicate rejected with error	Duplicate rejected	Pass
TC004	Login with valid credentials	User authenticated and token returned	Login successful, token returned	Pass
TC005	Login with invalid credentials	Login rejected with error	Error response returned	Pass
TC006	Protected API without token	Access denied (unauthorised)	Access denied	Pass
TC007	Protected API with valid JWT	Token verified, access allowed	Endpoint accessed successfully	Pass
TC008	Role-based authorization	Only role-permitted actions allowed	Unauthorised actions restricted	Pass
TC009	Create blog with valid data	Blog created and stored	Blog created and stored	Pass
TC010	Create blog with invalid data	Error returned, no record created	Validation response returned	Pass
TC011	Retrieve blog posts	Posts returned in structured format	Records retrieved	Pass
TC012	Update existing blog	Blog updated in MongoDB	Blog updated	Pass

ID	Scenario	Expected Result	Actual Result	Status
TC013	Delete existing blog	Blog removed from retrieval	Blog deleted	Pass
TC014	Add comment to blog	Comment linked to the blog	Comment added	Pass
TC015	AI blog content generation	Relevant content returned	Content returned	Pass
TC016	AI generation with empty prompt	Validation error returned	Empty prompt rejected	Pass
TC017	AI content summarisation	Short summary returned	Summary returned	Pass
TC018	MongoDB data storage check	Data stored correctly in collection	Record found with expected data	Pass
TC019	Invalid API endpoint	Route-not-found error	Proper error received	Pass
TC020	Server error handling	Error handled without crash	Structured error response returned	Pass

12.4 Result Summary

Category	Test Cases	Passed
Authentication and Authorization (TC001 to TC008)	8	8
Blog Management (TC009 to TC013)	5	5
Comments (TC014)	1	1
AI Features (TC015 to TC017)	3	3
Database (TC018)	1	1
API and Error Handling (TC019, TC020)	2	2
Total	20	20

12.5 Bug Tracking

No critical bugs were identified during testing. All tested functions produced the expected results.

12.6 Sign-Off

Tester	Date
Padmashri V	23/09/2026

13. API Endpoint Verification

13.1 Route Summary

Method	Endpoint	Access	Expected Status
POST	/api/auth/register	Public	201 Created
POST	/api/auth/login	Public	200 OK (returns JWT)
GET	/api/blogs	Public	200 OK (list of blogs)
POST	/api/blogs	Authenticated user	201 Created
POST	/api/ai/generate-blog	Authenticated user	201 Created
POST	/api/ai/summarize	Authenticated user	200 OK

13.2 Sample Execution

Create blog: POST /api/blogs (Status 201 Created, 59 ms)

```
// Request
{ "title": "My First Blog", "content": "This is my first blog content",
  "category": "Technology" }

// Response
{
  "_id": "6a44b6ffced94af06fb0bf90",
  "title": "My First Blog",
  "content": "This is my first blog content",
  "category": "Technology",
  "author": "6a44b49bcd94af06fb0bf8c",
  "authorName": "rohan",
  "createdAt": "2026-07-01T06:43:11.781Z",
  "updatedAt": "2026-07-01T06:43:11.781Z"
}
```

AI summary: POST /api/ai/summarize (Status 200 OK, 5.67 s)

```
// Request
{ "content": "React.js is a popular JavaScript library used for building
dynamic UIs..." }

// Response
{
  "summary": "React.js is a popular JavaScript library used for building
dynamic and interactive user interfaces (UIs) for web applications. Its core
concept is a component-based architecture."
}
```

13.3 Performance Summary

Measure	Result
Average throughput	240 requests per minute (local Express runtime)
MongoDB connection latency	12 ms (Atlas cloud cluster)
Overall verification	12 of 12 integration tests passed

14. Conclusion and Future Scope

AI BlogNest meets its goal of making blog creation and management simpler, safer and faster. It combines a layered Node.js and Express backend, MongoDB storage, JWT-based role control and Google Gemini AI in one API. All 20 UAT test cases passed, and the API responded quickly in local testing.

Future improvements:

- Build a React frontend for the API.
- Add AI-based tag and category suggestions.
- Add image upload and media storage.
- Add pagination, full-text search and post analytics.
- Deploy the backend and database on a cloud platform.

15. Technology Summary

Area	Technologies
Client	Postman, Thunder Client, JSON over REST
Backend	Node.js, Express.js
Database	MongoDB, Mongoose, MongoDB Atlas
Security	JWT, bcrypt.js, role-based access control, validation, sanitisation, rate limiting
AI	Google Gemini generative model (draft generation, summarisation, content assistance)
Tools	Visual Studio Code, npm, localhost server